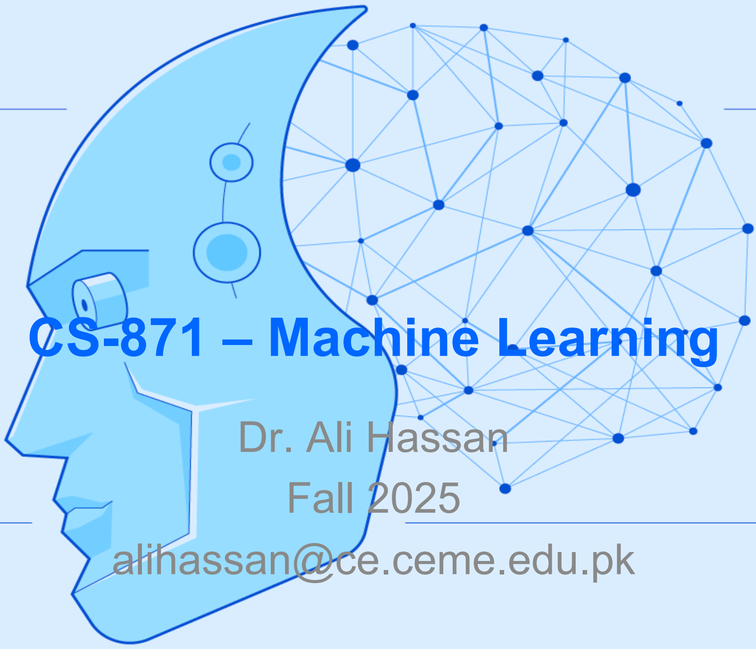


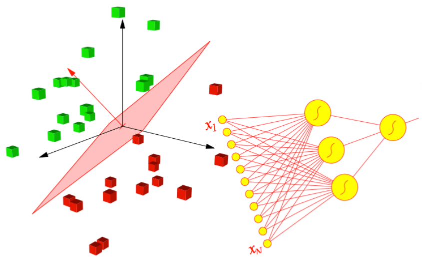
CS-871 – Machine Learning

Dr. Ali Hassan
Fall 2025
alihassan@ce.ceme.edu.pk

1

Announcements

- Assignment 2 Ready
  - Deadline 20 Oct 2024



2

Outline

- Previous Lecture
  - Linear Regression with One Variable
  - Gradient Descent (GD)

Todays Lecture

- This Lecture
  - Gradient Descent
    - Batch GD
    - Stochastic GD
  - Linear Regression with Multiple Variables
  - Linear Regression with Normal Equations
  - Python Code

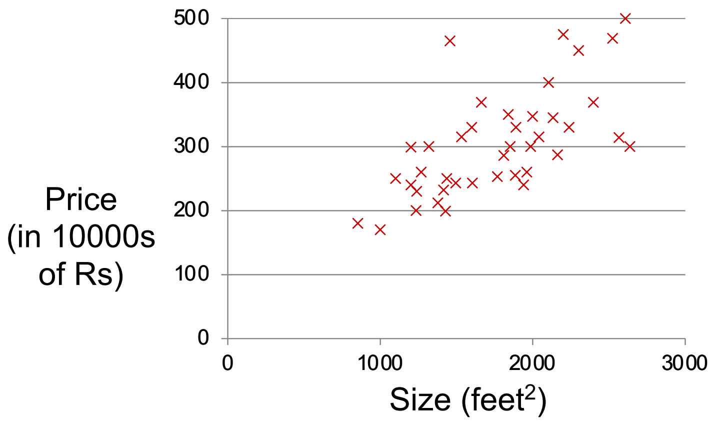
RECAP

5

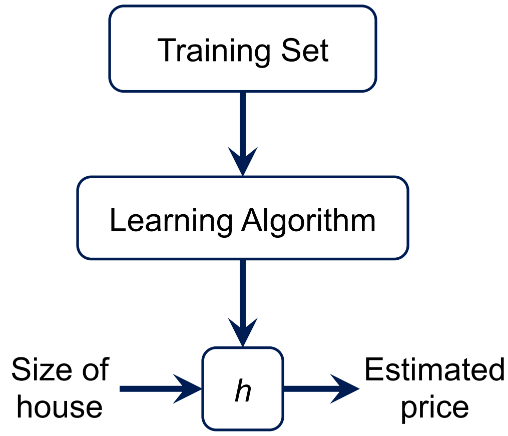
Housing Prices

<u>Supervised Learning</u>
Given the "right answer" for each example in the data.

<u>Regression Problem</u>
Predict real-valued or continuous output



6



Function or Hypothesis

Dr. Ali Hassan, DC&SE, College of E&ME, NUST 7

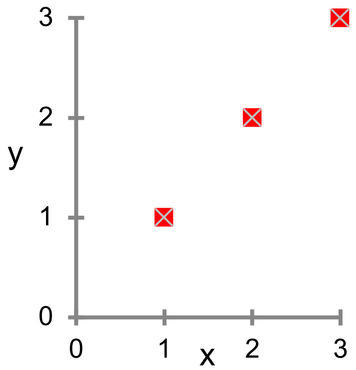
7

Hypothesis and Cost function

Dr. Ali Hassan, DC&SE, College of E&ME, NUST 8

8

Relationship bw J and h



Dr. Ali Hassan, DC&SE, College of E&ME, NUST 9

9

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Dr. Ali Hassan, DC&SE, College of E&ME, NUST 10

10

Take Home Message

- Minimising $J(\theta)$ will find the best fitting $h_\theta(x)$

OR

Value of θ that minimises the cost function

will

minimise the prediction error

11

Gradient descent algorithm | Linear Regression Model

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(for $j = 1$ and $j = 0$)

}

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

$$h_\theta(x) = \theta_0 + \theta_1 x$$

12

Gradient descent algorithm

repeat until convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x^{(i)}$$

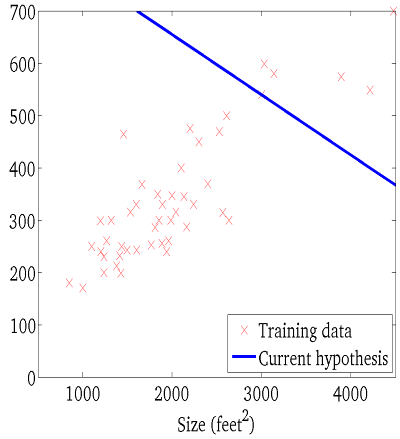
}

update θ_0 and θ_1 simultaneously

13

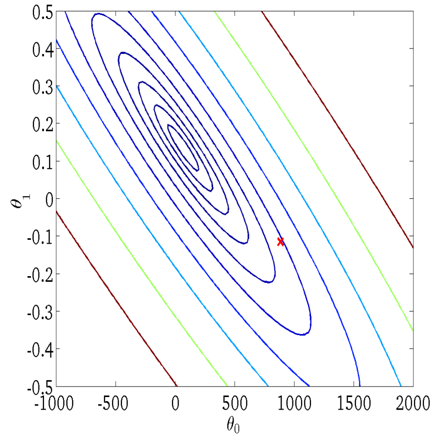
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

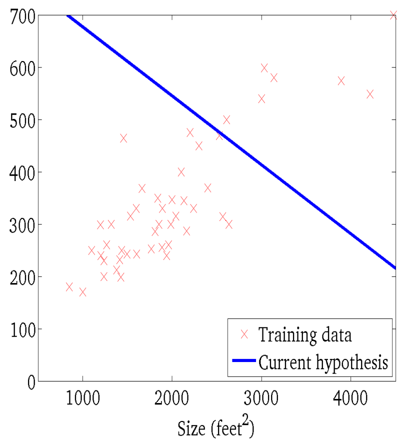
(function of the parameters θ_0, θ_1)

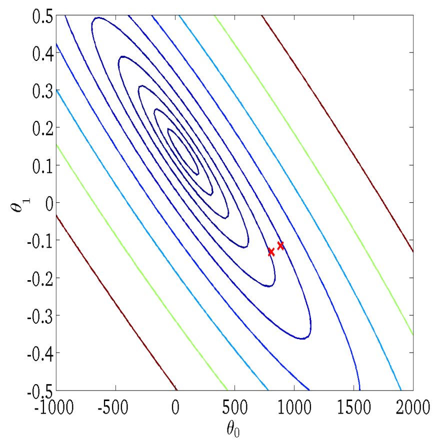


14

$h_\theta(x)$
(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$
(function of the parameters θ_0, θ_1)

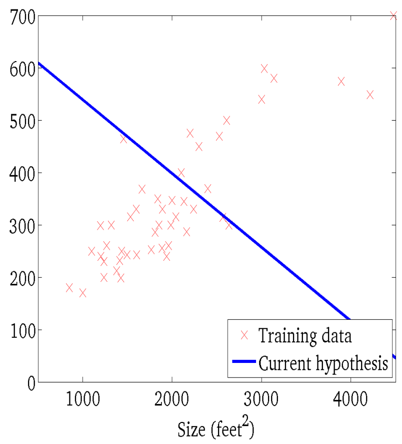


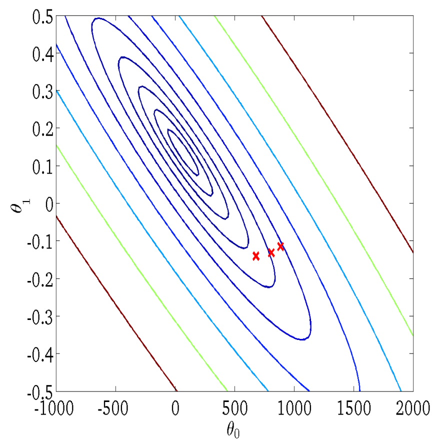


15

$h_\theta(x)$
(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$
(function of the parameters θ_0, θ_1)





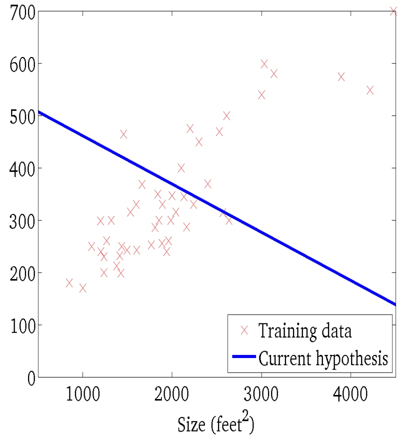
16

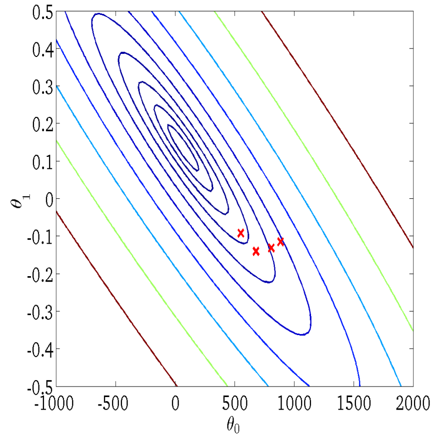
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)





Dr. Ali Hassan, DC&SE, College of E&ME, NUST 17

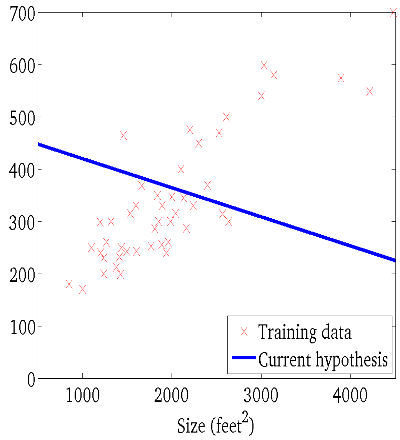
17

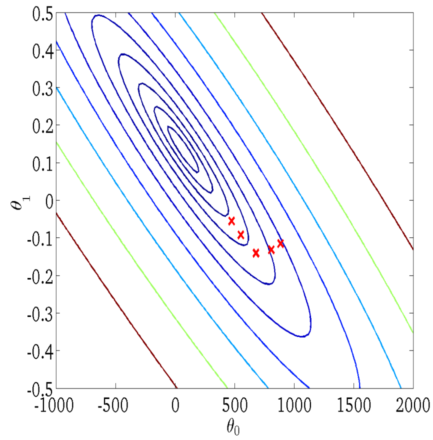
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)





Dr. Ali Hassan, DC&SE, College of E&ME, NUST 18

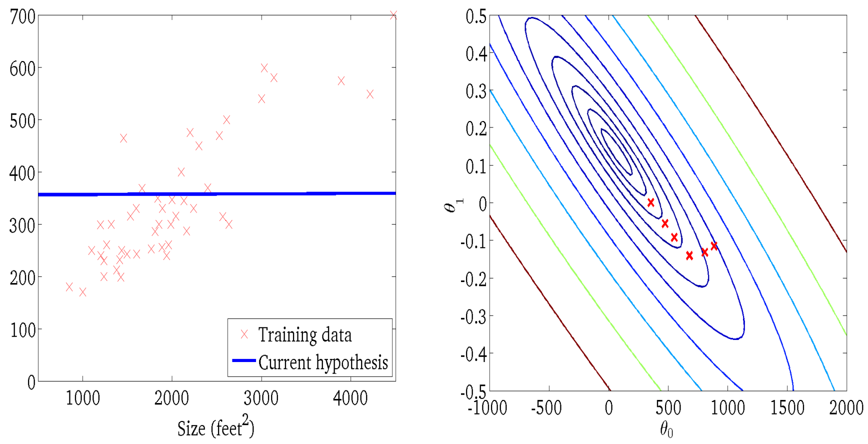
18

$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



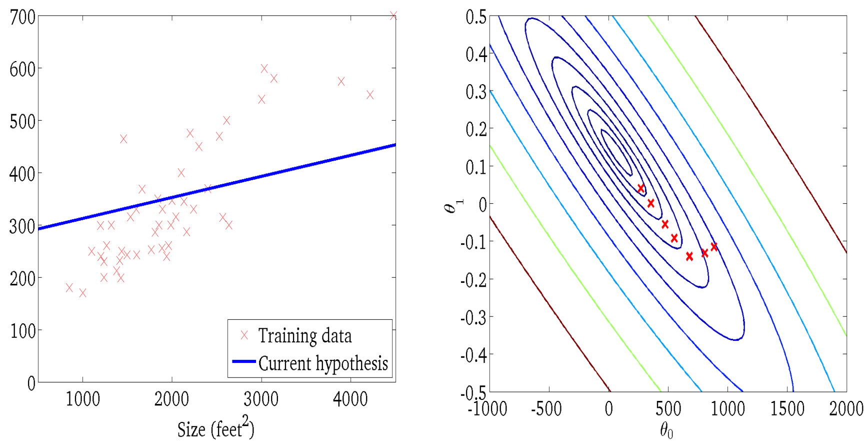
19

$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

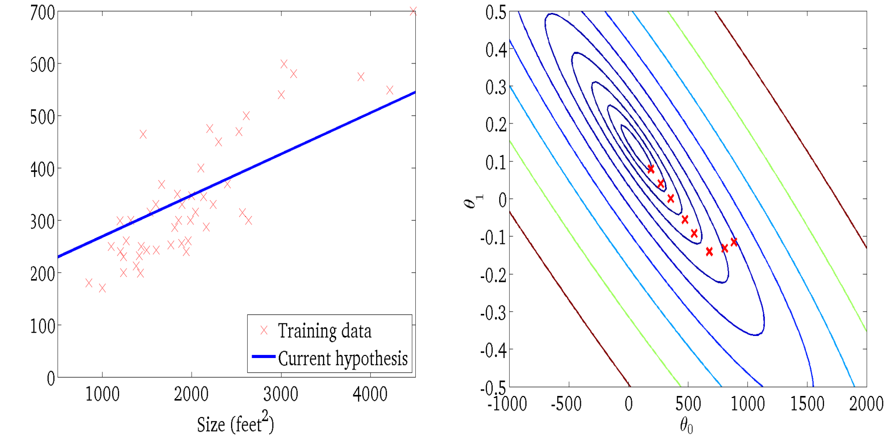
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



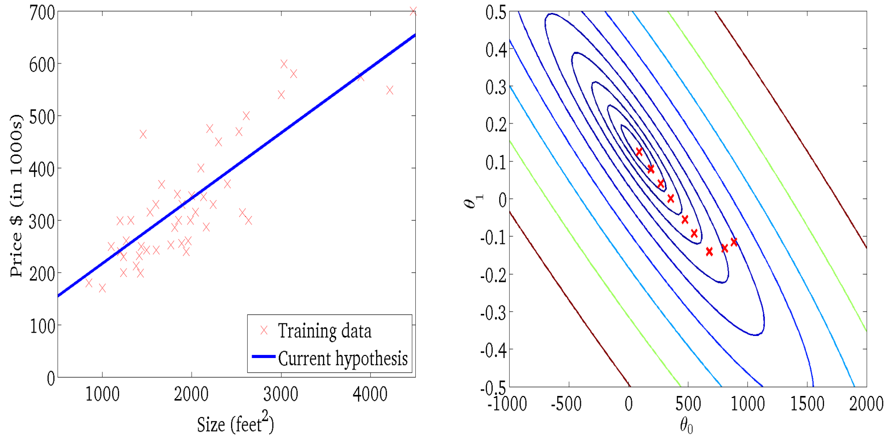
20

$h_\theta(x)$	$J(\theta_0, \theta_1)$
(for fixed θ_0, θ_1, this is a function of x)	(function of the parameters θ_0, θ_1)



21

$h_\theta(x)$	$J(\theta_0, \theta_1)$
(for fixed θ_0, θ_1, this is a function of x)	(function of the parameters θ_0, θ_1)



22

Linear regression with multiple variable

GRADIENT DESCENT

Single features (variables)

Size (feet^2) x	Price ($1000) y
2104	460
1416	232
1534	315
852	178
…	…

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Multiple features (variables)

Size (feet^2)	Number of bedrooms	Number of floors	Age of home (years)	Price
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

Notation:

n = number of features

$x^{(i)}$ = input (features) of i^{th} training example.

$x_j^{(i)}$ = value of feature j in i^{th} training example.

Explain how we make matrices

- Hypothesis:
  - Previously: $h_\theta(x) = \theta_0 + \theta_1 x$
    - Uni-variate as only one x
  - Now:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

    - Multi-variate linear regression

$$h_\theta(x) = \theta^T x$$

How

Hypothesis: $h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \ldots, \theta_n)$$

$\}$ (simultaneously update for every) $j = 0, \ldots, n$

30

Gradient Descent

Previously (n=1):

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

$\}$

New algorithm $(n \geq 1)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \ldots, n$)

$\}$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

$\ldots$

31

Feature Scalling

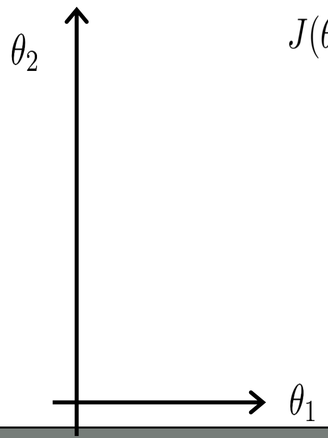
GRADIENT DESCENT

Feature Scaling

Idea: Make sure features are on a similar scale.

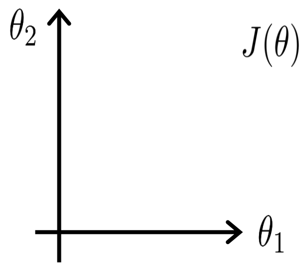
E.g. x_1 = size (0-2000 feet^2)

x_2 = number of bedrooms (1-5)



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$



Feature Scaling

Get every feature into approximately a range.

$$-1 \le x_i \le 1$$

Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean (Do not apply to $x_0 = 1$).

E.g. $x_1 = \frac{size-1000}{2000}$

$x_2 = \frac{\#bedrooms-2}{5}$

$$-0.5 \le x_1 \le 0.5, -0.5 \le x_2 \le 0.5$$

Learning Rate

DEBUGGING GRADIENT DESCENT

36

36

Gradient descent

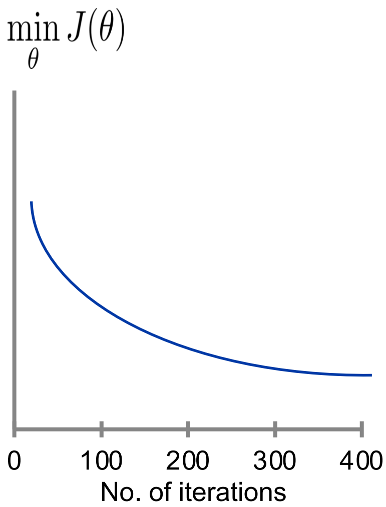
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- "Debugging": How to make sure gradient descent is working correctly.
- How to choose learning rate α.

Dr. Ali Hassan, DC&SE, College of E&ME, NUST 37

37

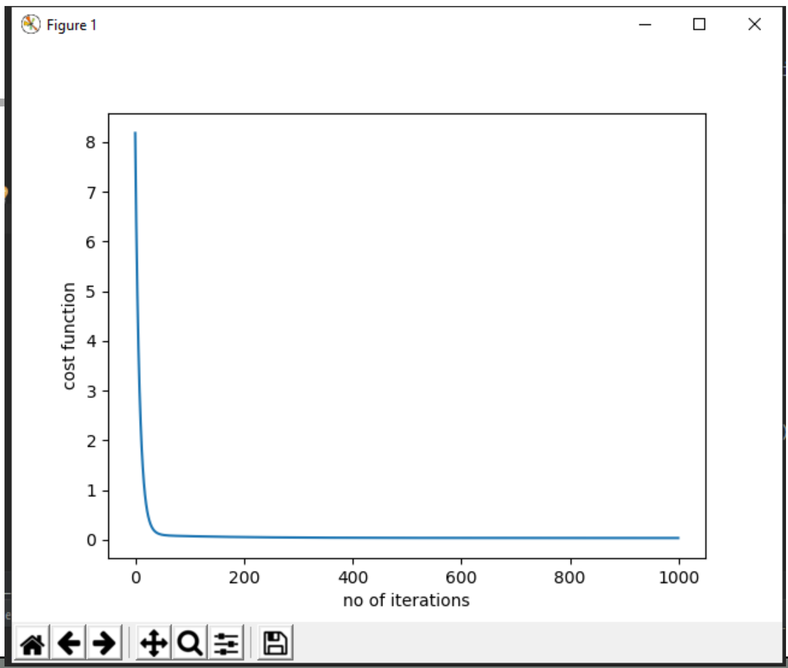
Making sure gradient descent is working correctly.



Example automatic convergence test:

Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

38



39

Making sure gradient descent is working correctly

- For sufficiently small α,
  - $J(\theta)$ should decrease on every iteration.
- But if α is too small,
  - gradient descent can be slow to converge
- Gradient descent not working
  - Use smaller α

Summary

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

- To choose α , try

$$\ldots, 0.001, \quad , 0.01, \quad , 0.1, \quad , 1, \ldots$$

BATCH GRADIENT DESCENT

42

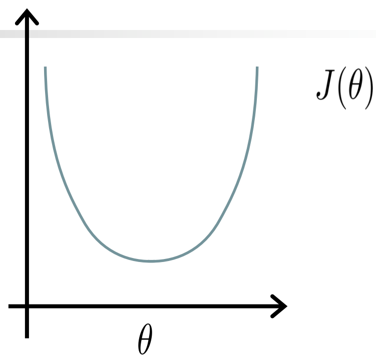
STOCHASTIC GRADIENT DESCENT

43

Normal Equations

LINEAR REGRESSION WITH MULTIPLE VARIABLES

Gradient Descent

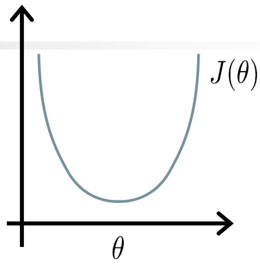


Normal equation: Method to solve for θ analytically.

Intuition: If 1D $(\theta \in \mathbb{R})$

$$J(\theta) = a\theta^2 + b\theta + c$$

Take derivative wrt θ and put it equal to zero.
Solve for θ



$\theta \in \mathbb{R}^{n+1}$ $\quad J(\theta_0, \theta_1, \ldots, \theta_m) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$

$\frac{\partial}{\partial \theta_j} J(\theta) = \cdots = 0$ (for every j)

Solve for $\theta_0, \theta_1, \ldots, \theta_n$

Examples: $m = 4$.

x_0	Size (feet²) x_1	Number of bedrooms x_2	Number of floors x_3	Age of home (years) x_4	Price ($1000) y
1	2104	5	1	45	460
1	1416	3	2	40	232
1	1534	3	2	30	315
1	852	2	1	36	178

m **examples** $(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})$; n **features.**

$$x^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix} \in \mathbb{R}^{n+1}$$

E.g. If $x^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \end{bmatrix}$

$$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix} \qquad y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$$

$$\theta = (X^T X)^{-1} X^T y$$

Matlab: θ = pinv(X'*X)*X'*y

Python: theta_best = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)

Derivation of Normal Equations for Linear Regression

Instructor: Dr. Ali Hassan

1 Linear Regression

In these notes I want to explain you how the normal equations for linear regression are derived. For doing this, I would be using the vector and matrix calculus. Please do remember that you can find these details on the internet on different websites like wikipedia. The idea is same but the symbols might be different. I am going to stay consistant with my course contents.

We have defined the following hypothesis function:

$$h_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \ldots + \theta_n x_n, \tag{1}$$

where each x is a n-dimensional vector or observation. Our goal is to minimising the cost function $J(\Theta)$ using least squares as follows:

m training examples, n features

Gradient Descent	Normal Equation
• Need to choose α .	• No need to choose α
• Needs many iterations.	• Don't need to iterate.
• Works well even when n is large.	• Need to compute $(X^T X)^{-1}$
	• Slow if n is very large.

Recap

- Hypothesis:
  - Uni-variate as only one x

$$h_\theta(x) = \theta_0 + \theta_1 x$$

  - Multi-variate linear regression

$$h_\theta(x) = \theta^T x$$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Recap

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Goal: $\min_{\Theta} J(\Theta)$

Recap

- Minimising $J(\theta)$ will find the best fitting $h_\theta(x)$

OR

Value of θ that minimises the cost function

will

minimise the prediction error

54

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \ldots, \theta_n)$$

} (simultaneously update for every) $j = 0, \ldots, n$

55

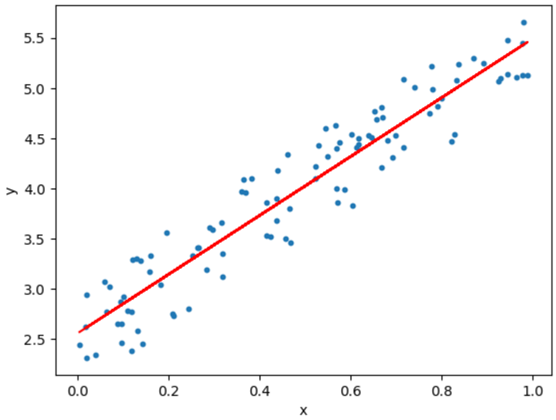
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def computeCost(X, y, theta):
    temp1 = 0
    h = X.dot(theta)
    temp1 = 1 / (2 * m) * np.sum(np.square(h - y))
    return temp1

def gradientDescent(X, y, theta, alpha, iterations):
    J_history = np.zeros(iterations)
    for iter in range(iterations):
        temp = np.dot(X, theta) - y
        temp = np.dot(X.T, temp)
        theta = theta - (alpha/m) * temp
        J_history[iter] = computeCost(X, y, theta)
    return (theta, J_history)
```


56

POLYNOMIAL REGRESSION

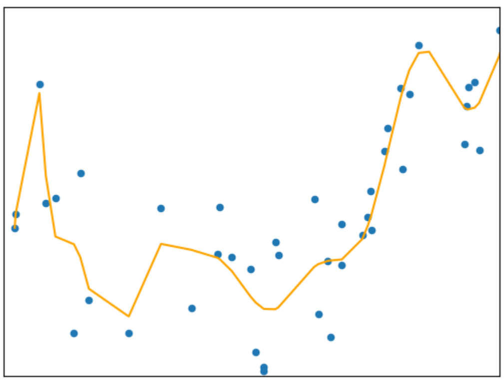
57

- Linear regression requires the relation between the dependent variable (y) and the independent variable (x) to be linear



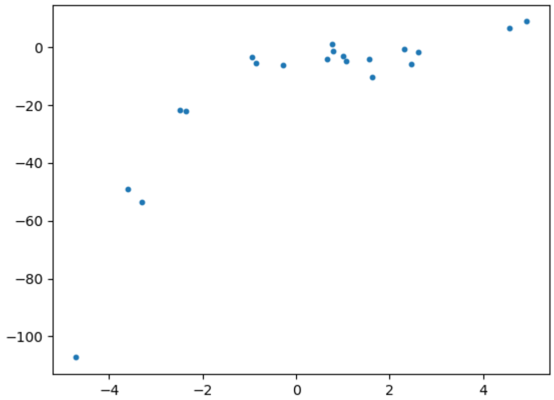
58

- What if the distribution of the data was non-linear

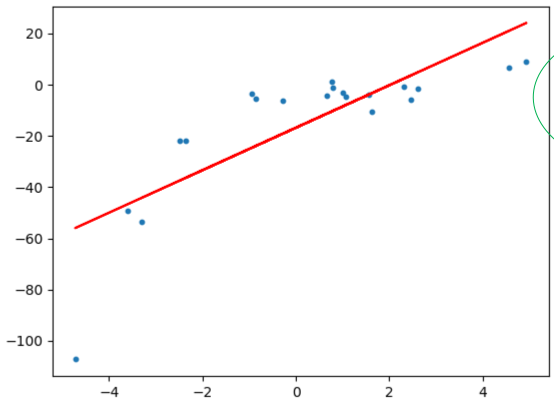


59

Need for Polynomial Reg



Linear Fit



Under-Fitting

RMSE of linear regression is **15.908242501429998.**
R^2 score of linear regression is **0.6386750054827146**

Solution

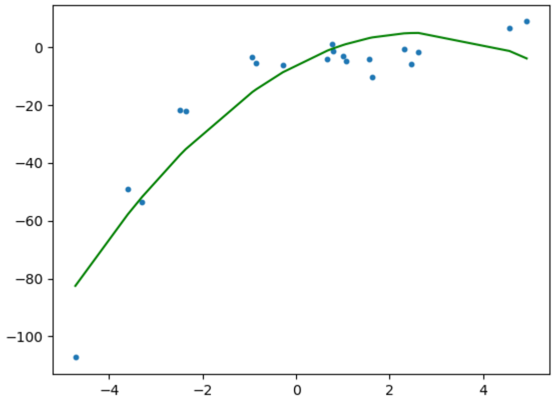
- To overcome under-fitting, we need to increase the complexity of the model
- Generate a higher order equation by adding powers of the original features as new features

- Linear Equations: $y = \theta_0 + \theta_1 x$
- Quadratic Equation: $y = \theta_0 + \theta_1 x + \theta_2 x^2$

Question

- Will we still consider this linear model??
  - $y = \theta_0 + \theta_1 x + \theta_2 x^2$
- as the coefficients/weights associated with the features are still linear.
- x^2 is only a feature.
- However the curve that we are fitting is **quadratic** in nature

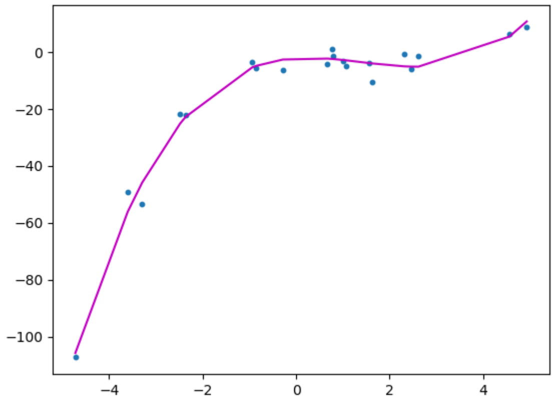
Quadratic Fitting



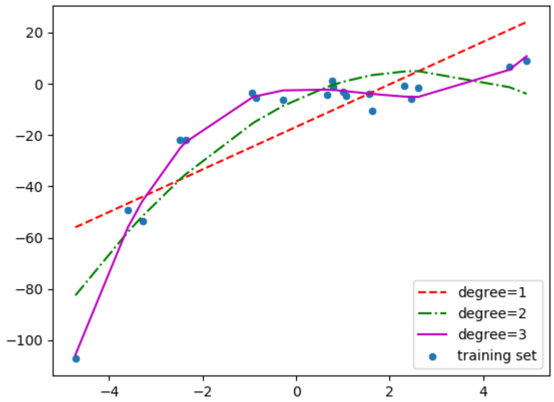
RMSE of polynomial regression is **10.120437473614711.**
R^2 of polynomial regression is **0.8537647164420812.**

64

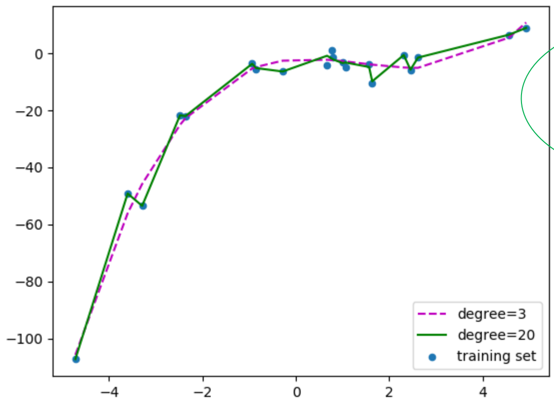
Cubic Fit



65



Polynomial of Degree 20



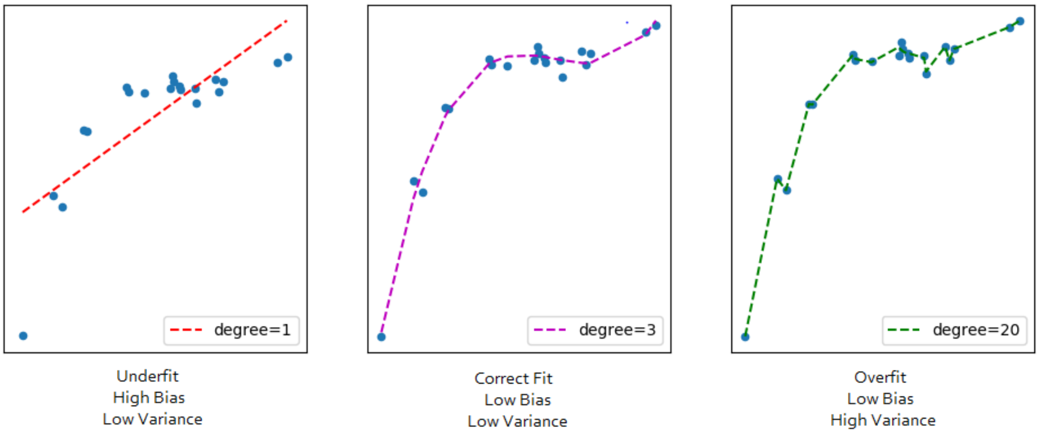
Over-Fitting

Solution

- How do we choose an optimal model
- Understand the bias vs variance trade-off

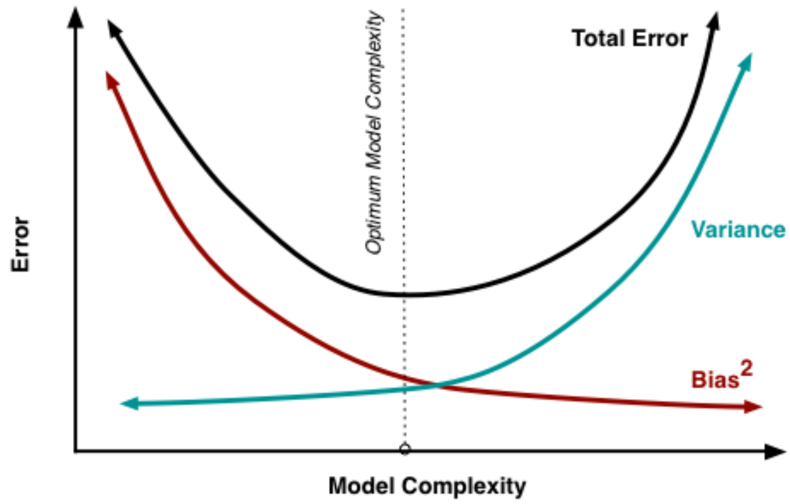
The Bias vs Variance trade-off

- **Bias** refers to the error due to the model's simplistic assumptions in fitting
  - high bias means model is unable to capture the patterns in the data and results in **under-fitting**.
- **Variance** refers to the error due to the complex model trying to fit the data
  - High variance means the model passes through most of the data points and it results in **over-fitting** the data.



Dr. Ali Hassan, DC&SE, College of E&ME, NUST 70

70



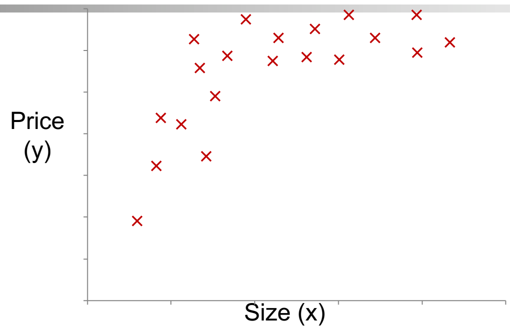
Dr. Ali Hassan, DC&SE, College of E&ME, NUST 71

71

Housing prices prediction

$$h_\theta(x) = \theta_0 + \theta_1 \times frontage + \theta_2 \times depth$$

Polynomial regression



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

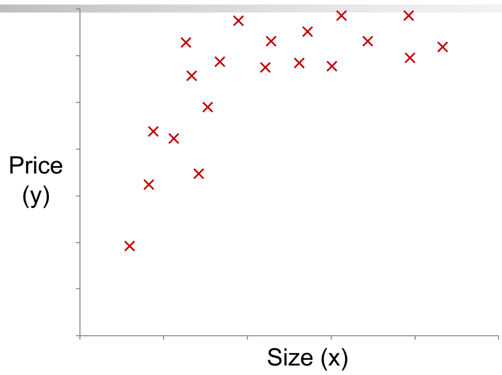
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

$$\begin{aligned} h_\theta(x) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \\ &= \theta_0 + \theta_1 (size) + \theta_2 (size)^2 + \theta_3 (size)^3 \end{aligned}$$

$x_1 = (size)$ $\quad x_3 = (size)^3$

$x_2 = (size)^2$

Choice of features



$$h_\theta(x) = \theta_0 + \theta_1(size) + \theta_2(size)^2$$

$$h_\theta(x) = \theta_0 + \theta_1(size) + \theta_2\sqrt{(size)}$$

74

Uptill Now

- Gradient Descent—Feature Scaling
- Linear Regression with Multiple Variables
- Linear Regression -- Normal Equations

Next Topic

- Logistic Regression

75

Next Lecture

- Logistic Regression
- Reading Work
  - Chapter 7 Linear Regression from
    - *Machine Learning: A Probabilistic Perspective* by Kevin Murphy
    - Section 7-7.5.1 + 7.5.4
  - Reading Notes

Classification

LOGISTIC REGRESSION

Classification

- Email: Spam / Not Spam?
- Online Transactions:
  - Fraudulent (Yes / No)?
- Tumor: Malignant / Benign ?

$y \in \{0, 1\}$

0: "Negative Class" (e.g., benign tumor)
1: "Positive Class" (e.g., malignant tumor)

Supervise Learning: we know the correct answers
Classification: the outputs are discrete

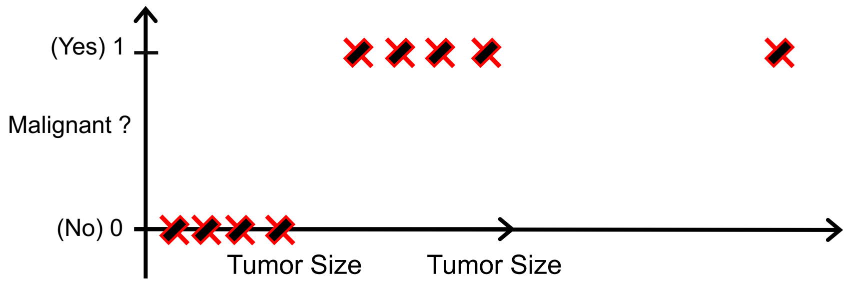
Logistic Regression

- Unlike Linear Regression, the dependent variable **(y)** can take a limited number of values only i.e, the dependent variable is **categorical**
- When the number of possible outcomes is only two it is called **Binary Logistic Regression**.

Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"



Restriction on Logistic Reg

- If we take the weighted sum of inputs as the output as we do in Linear Regression, the value can be more than 1
- we want a value between 0 and 1
- we don't output the weighted sum of inputs directly
- we pass it through a function that can map any real value between 0 and 1

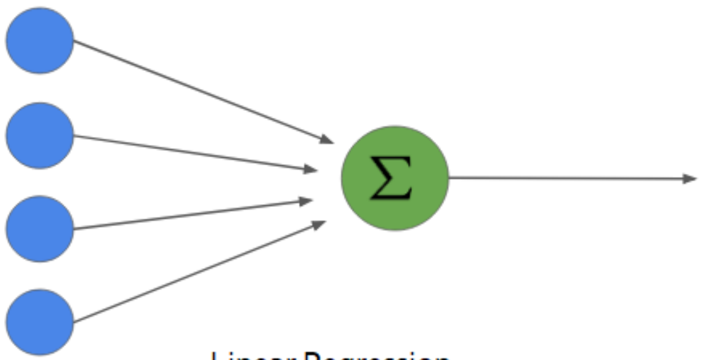
Restriction on LogReg

Classification: y = 0 or 1

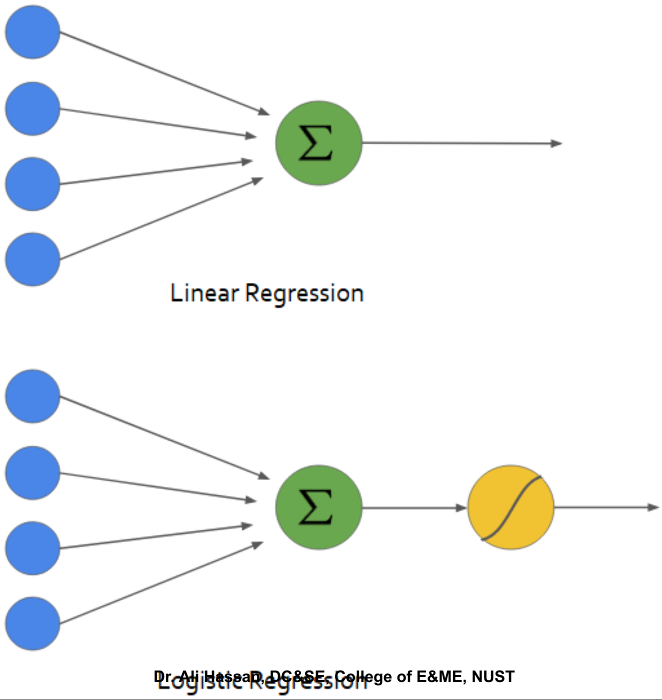
For linear regression:

$h_\theta(x)$ can be > 1 or < 0

Logistic Regression: $0 \leq h_\theta(x) \leq 1$



Linear Regression



84

ACTIVATION FUNCTIONS

85

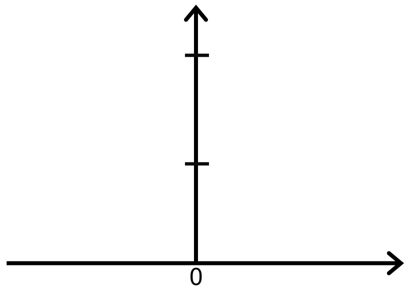
Activation Function

- $h_\theta(x) = \theta^T x$
- Pass this output through a limiting function called Activation function
- $g\left(h_\theta(x)\right) = g(\theta^T x)$
  - Where g is the activation function

Linear Activation Function

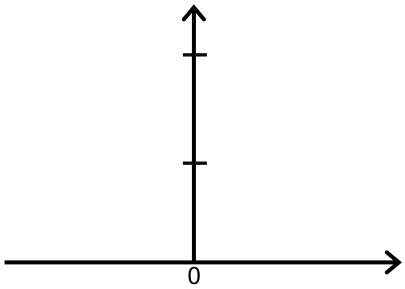
- $g(\theta^T x) = \theta^T x$

- No change to the output



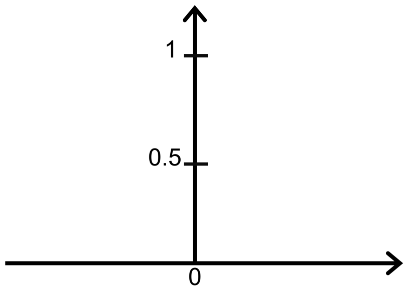
Rectified Linear Activation

- $g(\theta^T x) = reclin\ (\theta^T x)$
- $g(\theta^T x) = \max(0, \theta^T x)$



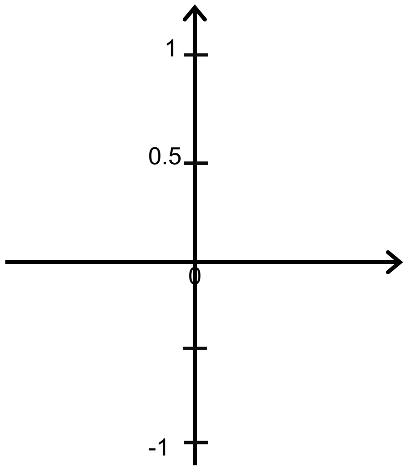
Sigmoid Activation

- $g(\theta^T x) = sigm(\theta^T x)$
- $= \frac{1}{1+\mathrm{e}^{(-\theta^T x)}}$
- $= \frac{1}{1+e^{-z}}$



Hyperbolic Tangent Activation

- $g(\theta^T x) = \tanh(\theta^T x)$
- $= \frac{\exp(\theta^T x) - \exp(-\theta^T x)}{\exp(\theta^T x) + \exp(-\theta^T x)}$
- $= \frac{\exp(2 * \theta^T x) - 1}{\exp(2 * \theta^T x) + 1}$



90

Hypothesis Representation

LOGISTIC REGRESSION

91

Logistic Regression Model

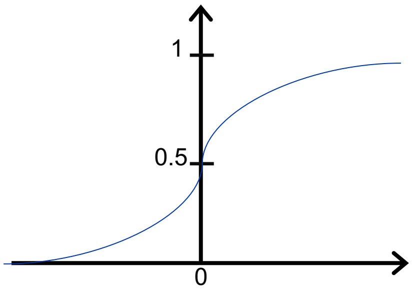
Want $0 \leq h_\theta(x) \leq 1$

$$h_\theta(x) = \theta^T x$$

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$

Sigmoid function
Logistic function



Sigmoid Func in Python

```python
def sigmoid(Z):
    return 1/(1+np.exp(-Z))
```

Interpretation of Hypothesis Output

$h_\theta(x)$ = estimated probability that y = 1 on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$$h_\theta(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant

$$h_\theta(x) = P(y = 1|x;\theta)$$

"probability that y = 1, given x, parameterized by θ "

$$P(y = 0|x;\theta) + P(y = 1|x;\theta) = 1$$
$$P(y = 0|x;\theta) = 1 - P(y = 1|x;\theta)$$

94

94

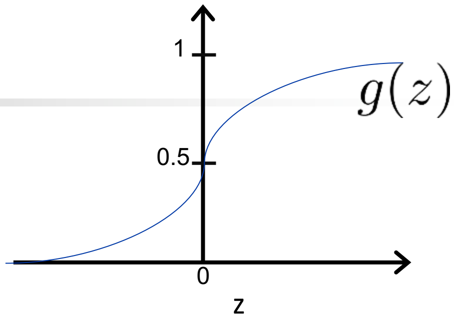
Decision Boundary

LOGISTIC REGRESSION

95

95

Logistic regression



$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$

Suppose predict "$y=1$" if $h_\theta(x) \geq 0.5$

θᵀx ≥ 0

g(z) ≥ 0.5
When z ≥ 0

and
g(x) < 0.5
When z< 0

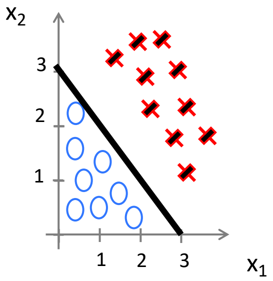
predict "$y=0$" if $h_\theta(x) < 0.5$

$\theta^T x < 0$

96

96

Decision Boundary



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Predict: y=1 if $\theta^T x \geq 0$

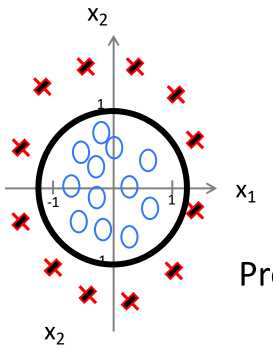
$$-3 + x_1 + x_2 \geq 0$$

97

97

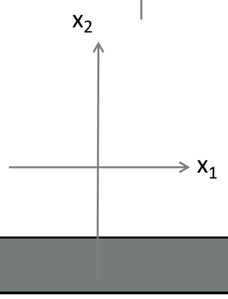
Non-linear decision boundaries

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

Predict "$y = 1$" if $-1 + x_1^2 + x_2^2 \geq 0$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \dots)$$

98

98

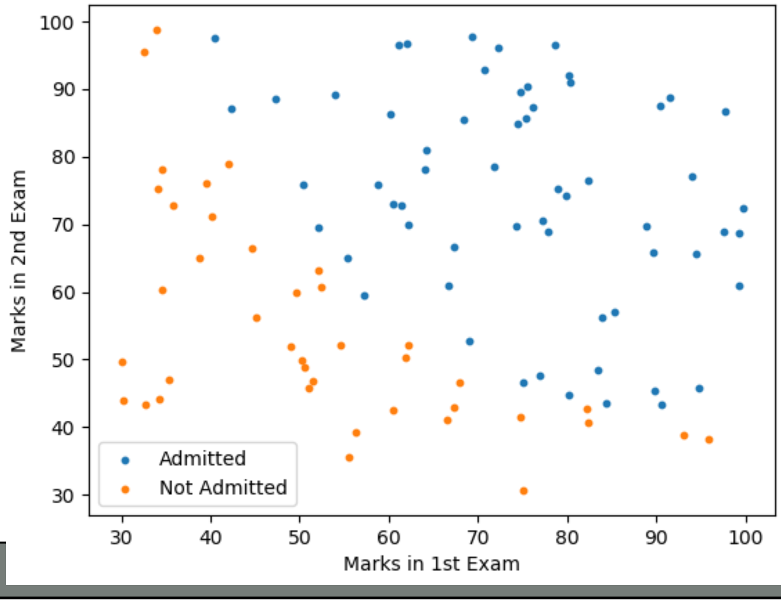
DataSet

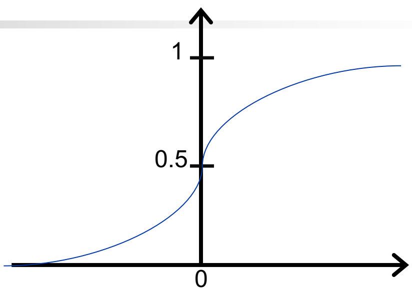
- The data consists of marks of two exams for 100 applicants.
- The target value takes on binary values 1,0.
  - 1 means the applicant was admitted to the university
  - 0 means the applicant didn't get an admission.

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

99

99





Cost Function

LOGISTIC REGRESSION

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

102

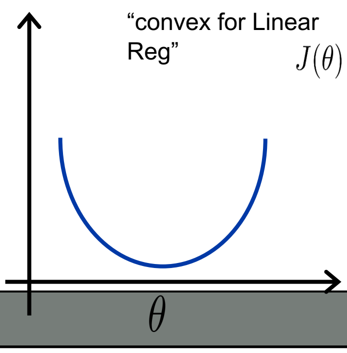
102

Cost function

Linear regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

$$\text{Cost}(h_\theta(x^{(i)}), y^{(i)}) = \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

"convex for Linear Reg"



"non-convex for Logistic Reg because of "

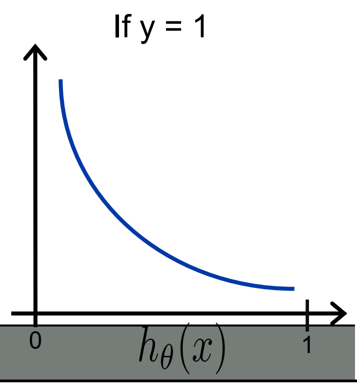
$$g(z) = \frac{1}{1+e^{-z}}$$

103

103

Logistic regression cost function

$$\mathrm{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

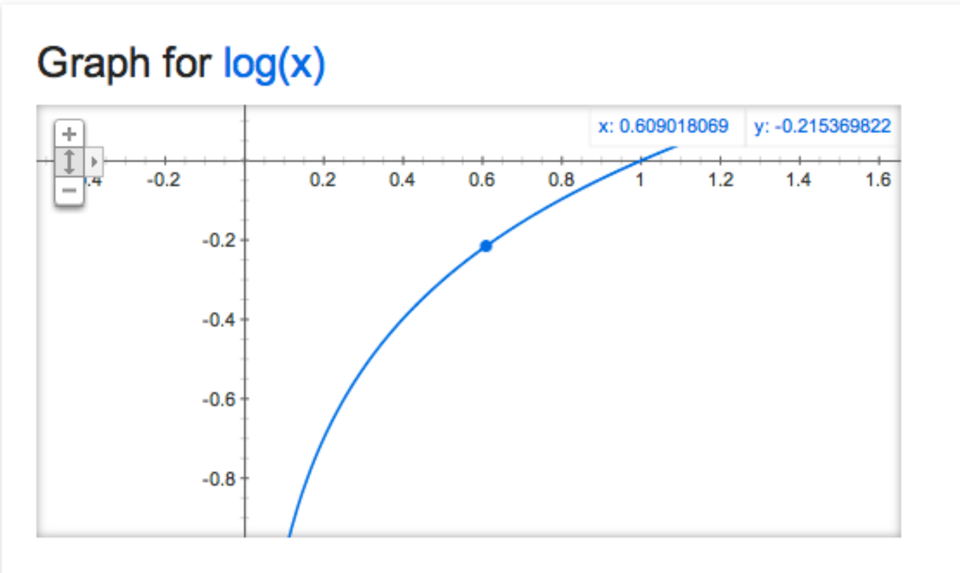


$\mathrm{Cost} = 0$ if $y = 1, h_\theta(x) = 1$
But as $h_\theta(x) \rightarrow 0$
$Cost \rightarrow \infty$

Captures intuition that if $h_\theta(x) = 0$, (predict $P(y = 1|x;\theta) = 0$), but $y = 1$, we'll penalize learning algorithm by a very large cost.

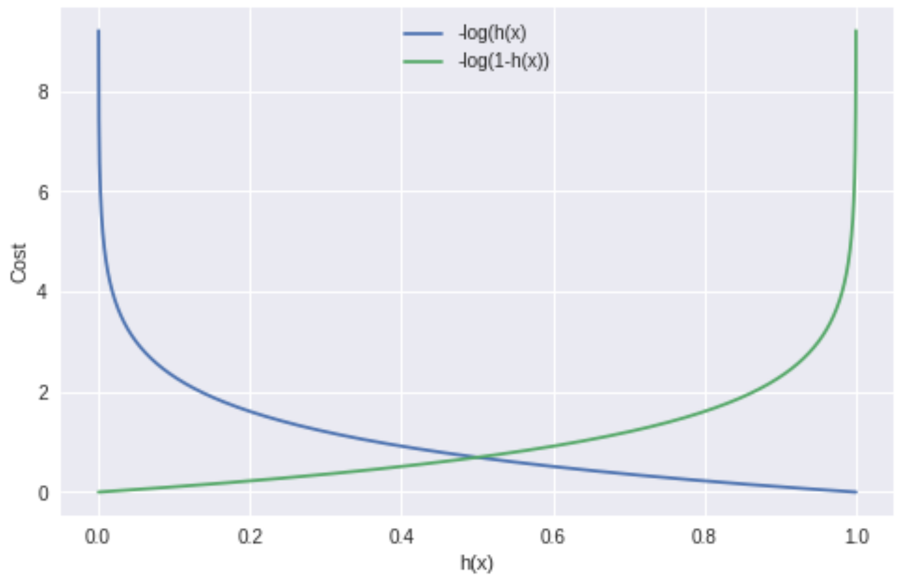
104

104



105

105



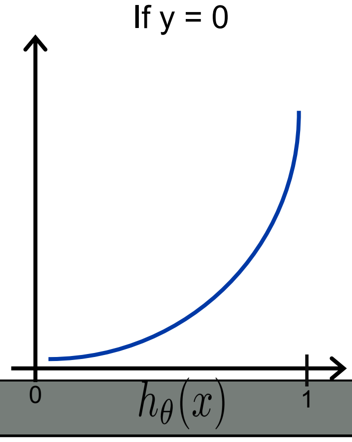
106

106

Logistic regression cost function

$$\mathrm{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

If y = 0



107

107

Simplified Cost Function and Gradient Descent

LOGISTIC REGRESSION

108

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\mathrm{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\mathrm{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

We can combine both of equations using:

$$cost(h(x), y) = -ylog(h(x)) - (1-y)log(1-h(x))$$

109

Logistic regression cost function

- The cost for all the training examples denoted by *J(θ)* can be computed by taking the average over the cost of all the training samples

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^i log(h(x^i)) + (1 - y^i) log(1 - h(x^i))]$$

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \mathrm{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))]$$

To fit parameters θ :

$$\min_\theta J(\theta)$$

To make a prediction given new x:

Output $h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

(simultaneously update all θ_j)

}

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} (h(x^i) - y^i)\ x_j^i$$

112

112

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h(x^i) - y^i)\ x_j^i$$

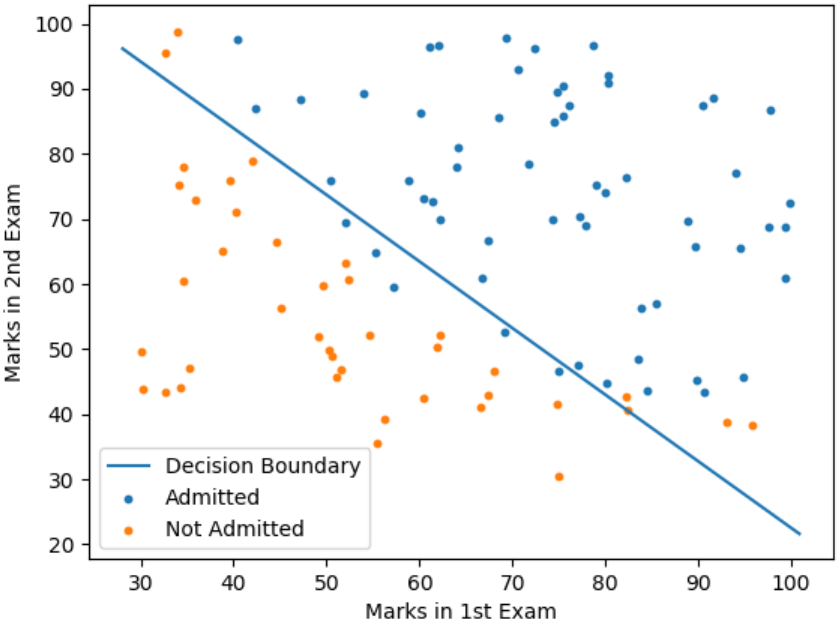
(simultaneously update all θ_j)

}

Algorithm looks identical to linear regression!

113

113



114

114

Complete Derivative of Logistic function

Machine Learning – Lecture Notes

Instructor: Dr. Ali Hassan *Date Created: 28 Nov 2016*

1 Derivation of Logistic Regression Cost Function

The cost function of the logistic regression is given by:

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)}\log h_\theta(x^{(i)}) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{i=1}^{n}\theta_j^2 \quad (1)$$

we know that run the gradient descent, we need to apply to following equation:

$$\theta_j := \theta_j - \alpha\frac{\partial}{\partial\theta}J(\Theta) \quad (2)$$

where we need to plug in the $J(\Theta)$. For this derivation lets drop the regularser term and see what happens.

$$h_\theta(x^{(i)}) = \frac{1}{1+e^{-\theta^T x}} \quad (3)$$

$$g(z) = \frac{1}{1+e^{-z}} \quad (4)$$

The derivative of $g(z)$ is given by:

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

$$g'(z) = \frac{e^{-z}}{\ldots} \quad (5)$$

115

115

Announcements

- Assignment 2 Uploaded
- Reading Assignment
  - Logistic Regression
  - Reading: Chapter 8
    - Kevin Murphy

116

END OF CLASS

117